

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Vasilean Vitalie
Group FAF-241

Report

Laboratory Work No. 5
Operating Systems
Topic: The File System with Your Own Hands (Linux)

Checked by:
univ. assist. / lect. Patricia Reițman
Department of Software Engineering and Automation
FCIM, UTM

Chisinau – 2026

Contents

1 INTRODUCTION AND GOAL **3**

1.1 Introduction 3

1.2 Lab Objective 3

2 THE TREE AND FILE TYPES **4**

2.1 Command Execution and Raw Output 4

2.2 Analysis and Observations 4

3 INODES AND LINKS **5**

3.1 Command Execution and Raw Output 5

3.2 Analysis and Observations 5

4 PERMISSIONS AND SPECIAL BITS **7**

4.1 Command Execution and Raw Output 7

4.2 Analysis and Observations 7

5 DISKS AND THE KERNEL AS FILES **9**

5.1 Command Execution and Raw Output 9

5.2 Analysis and Observations 9

6 STRETCH GOALS **10**

6.1 Stretch Goal A: Magic Numbers 10

6.2 Stretch Goal C: Deletion of an Active Open File 10

CONCLUSION **11**

REFERENCES **12**

1 INTRODUCTION AND GOAL

1.1 Introduction

Operating systems provide high-level abstractions to manage persistent secondary storage, device communication, and running process states. Under the UNIX and Linux design philosophy, these resources are unified under a hierarchical virtual filesystem tree. Rather than interacting directly with block addresses on disk hardware, user applications access files, directories, character interfaces, storage devices, and kernel tables through standardized POSIX system call abstractions.

1.2 Lab Objective

The primary objective of this laboratory work is to investigate the foundational mechanisms of the Linux filesystem using direct command-line utilities and kernel inspections. The scope covers:

- the filesystem tree and file type classification from file metadata;
- inode structures, directory entry namespaces, hard links, and symbolic links;
- file size semantics versus allocated physical disk space (sparse files);
- Discretionary Access Control (DAC) permissions, directory traversal rules, umask operations, and special permission bits (Setuid, Sticky bit);
- disk partitioning, mount points, and pseudo-filesystems exposing kernel internal state (`/proc`).

2 THE TREE AND FILE TYPES

2.1 Command Execution and Raw Output

The system directory root and core entry points were inspected using `ls`, `file`, and `du`:

```
vitalie@LinuxLabs:~/os-lab5$ ls -ld /etc /etc/passwd /bin /dev/null /tmp
vitalie@LinuxLabs:~/os-lab5$ du -sh /var/log 2>/dev/null ; du -sh /usr 2>/dev/null
74M     /var/log
6.3G    /usr
```

2.2 Analysis and Observations

a) **File types from the initial character of `ls -ld`:**

- `/etc`: `d` → Directory.
- `/etc/passwd`: `-` → Regular file.
- `/bin`: `l` → Symbolic link.
- `/dev/null`: `c` → Character device.
- `/tmp`: `d` → Directory with sticky bit protection.

b) **Nature of `/bin`:** On modern Linux distributions, `/bin` is a symbolic link pointing to `/usr/bin`. This follows the UsrMerge architecture, consolidating all core binaries under `/usr`.

c) **Device classification in `/dev`:**

- Character device (`c`): `/dev/null` processes data as an unbuffered stream of single bytes.
- Block device (`b`): `/dev/sda` transfers data through the kernel page cache in fixed-size blocks.

d) **Disk usage comparison:** `/usr` occupies **6.3 GB**, whereas `/var/log` occupies **74 MB**. The `/usr` hierarchy is roughly 85 times larger because it houses core userland programs, shared libraries, and documentation, whereas `/var/log` only stores rotating system logs.

3 INODES AND LINKS

3.1 Command Execution and Raw Output

Hard links, symbolic links, directory structure links, and file allocation properties were evaluated:

```
vitalie@LinuxLabs:~/os-lab5$ stat notes.txt
  File: notes.txt
  Size: 6               Blocks: 8          IO Block: 4096   regular file
Device: 8,2     Inode: 1574449     Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/ vitalie)   Gid: ( 1000/ vitalie)

vitalie@LinuxLabs:~/os-lab5$ ln notes.txt hard.txt
vitalie@LinuxLabs:~/os-lab5$ ls -li notes.txt hard.txt
1574449 -rw-rw-r-- 2 vitalie vitalie 6 Oct 12 12:49 hard.txt
1574449 -rw-rw-r-- 2 vitalie vitalie 6 Oct 12 12:49 notes.txt

vitalie@LinuxLabs:~/os-lab5$ rm notes.txt
vitalie@LinuxLabs:~/os-lab5$ cat hard.txt ; ls -li hard.txt
hello
1574449 -rw-rw-r-- 1 vitalie vitalie 6 Oct 12 12:49 hard.txt

vitalie@LinuxLabs:~/os-lab5$ ln -s hard.txt soft.txt
vitalie@LinuxLabs:~/os-lab5$ ls -li hard.txt soft.txt
1574449 -rw-rw-r-- 1 vitalie vitalie 6 Oct 12 12:49 hard.txt
1576266 lrwxrwxrwx 1 vitalie vitalie 8 Oct 12 12:49 soft.txt -> hard.txt

vitalie@LinuxLabs:~/os-lab5$ rm hard.txt
vitalie@LinuxLabs:~/os-lab5$ cat soft.txt ; ls -l soft.txt
cat: soft.txt: No such file or directory
lrwxrwxrwx 1 vitalie vitalie 8 Oct 12 12:49 soft.txt -> hard.txt

vitalie@LinuxLabs:~/os-lab5$ mkdir d ; ls -ld d
drwxrwxr-x 2 vitalie vitalie 4096 Oct 12 12:49 d

vitalie@LinuxLabs:~/os-lab5$ mkdir d/x d/y ; ls -ld d
drwxrwxr-x 4 vitalie vitalie 4096 Oct 12 12:49 d

vitalie@LinuxLabs:~/os-lab5$ echo x > one.txt ; ls -l one.txt ; du -h one.txt
-rw-rw-r-- 1 vitalie vitalie 2 Oct 12 12:49 one.txt
4.0K    one.txt

vitalie@LinuxLabs:~/os-lab5$ truncate -s 1G big.img ; ls -lh big.img ; du -h big.img
-rw-rw-r-- 1 vitalie vitalie 1.0G Oct 12 12:49 big.img
0       big.img
```

3.2 Analysis and Observations

a) **Inode numbers and link reference counters:** The field `Inode: 1574449` specifies the unique inode index within filesystem device 8,2. The `Links: 2` parameter defines how many directory namespace entries point to that inode.

b) **Hard link data persistence:** Creating a hard link introduces a second directory entry sharing the identical inode (`1574449`). The command `rm notes.txt` invokes the `unlink()` system call, decrementing the link count from 2 to 1. Because the link count remains non-zero, the physical data blocks remain intact and accessible via `hard.txt`.

c) **Symbolic link behavior:** A symlink creates a new inode (`1576266`) holding the text string path `"hard.txt"`. When `hard.txt` is unlinked, reading `soft.txt` fails with `No such file or directory` because target path resolution fails. The size of `soft.txt` is exactly **8 bytes** because the ASCII string `"hard.txt"` consists of 8 bytes.

d) **Directory link counting:** A new directory `d` has **2 links**: the reference in the parent folder and its self-referential entry `d/.`. Adding subdirectories `d/x` and `d/y` increases the count to **4** because each subdirectory contains an entry `..` pointing back to `d`.

e) **Physical block allocation versus logical size:**

- `one.txt` contains 2 bytes (`'x'` and `\n`), but the ext4 filesystem allocates disk space in minimum block units of 4096 bytes (4.0 KB).
- `big.img` is a *sparse file* created via `truncate`. The kernel stores a logical size of 1.0 GB in the inode metadata without allocating physical disk extents until data is written, resulting in 0 bytes reported by `du`.

4 PERMISSIONS AND SPECIAL BITS

4.1 Command Execution and Raw Output

Standard permissions, directory traversal rules, umask calculation, and elevated bits were tested:

```
vitalie@LinuxLabs:~/os-lab5$ chmod 000 s.txt ; cat s.txt
cat: s.txt: Permission denied

vitalie@LinuxLabs:~/os-lab5$ chmod u+r s.txt ; cat s.txt
secret

vitalie@LinuxLabs:~/os-lab5$ umask ; touch new.txt ; mkdir newdir ; ls -ld new.txt newdir
0002
-rw-rw-r-- 1 vitalie vitalie    0 Oct 12 12:49 new.txt
drwxrwxr-x 2 vitalie vitalie 4096 Oct 12 12:49 newdir

vitalie@LinuxLabs:~/os-lab5$ chmod u=r,go= box ; ls box ; cat box/f
f
cat: box/f: Permission denied

vitalie@LinuxLabs:~/os-lab5$ chmod u=x,go= box ; ls box ; cat box/f
ls: cannot open directory 'box': Permission denied
hi

vitalie@LinuxLabs:~/os-lab5$ ls -l /usr/bin/passwd ; ls -ld /tmp
-rwsr-xr-x 1 root root 93640 Feb  2  2026 /usr/bin/passwd
drwxrwxrwt 15 root root   360 Oct 12 12:46 /tmp

vitalie@LinuxLabs:~/os-lab5$ cat /etc/shadow ; ls -l /etc/shadow
cat: /etc/shadow: Permission denied
-rw-r----- 1 root shadow 1335 Oct  5 13:09 /etc/shadow
```

4.2 Analysis and Observations

a) **Octal permission representations:**

- `600` (`rw-------`): Read and write access for owner only. Used for private cryptographic keys (e.g., `~/.ssh/id_rsa`).
- `644` (`rw-r--r--`): Owner read/write, world readable. Standard for configuration and web documents (e.g., `/etc/resolv.conf`).
- `755` (`rwxr-xr-x`): Full access for owner, read and execute for group/others. Standard for public binaries and system directories (e.g., `/usr/bin/ls`).
- `700` (`rwx------`): Full access restricted strictly to owner. Used for personal directories (e.g., `~/.ssh`).

b) **Ownership versus `chmod 000`:** Linux evaluates permission bits based on the matching credential tier. Since the user is the owner, the user class bits (`---`) apply directly, denying read access. File ownership grants the right to modify permission attributes via `chmod`, but does not grant automatic read privileges.

c) **Umask mechanics:** The active umask is `0002`.

$$\text{Base File Mode:} \quad 0666_8 \wedge \neg(0002_8) = 0664_8 \quad (\texttt{-rw-rw-r--})$$
$$\text{Base Directory Mode:} \quad 0777_8 \wedge \neg(0002_8) = 0775_8 \quad (\texttt{drwxrwxr-x})$$

d) **Directory permissions (`r` vs. `x`):**

- Read permission (`r`) allows reading the directory contents (listing directory entry names with `ls`). It does not permit resolving inodes or reading file content inside.
- Execute permission (`x`) allows path traversal and inode resolution. A known filename can be opened and read (`cat box/f`), but directory listings are forbidden (`ls box` fails).

e) **Special permission bits:**

- The `s` bit on `/usr/bin/passwd` is Setuid. It executes the program with the effective user ID of the file owner (`root`), allowing an unprivileged user to update their password hash inside `/etc/shadow`.
- The `t` bit on `/tmp` is the Sticky bit. In a world-writable directory, it restricts deletion or renaming of a file strictly to the file owner or root.

5 DISKS AND THE KERNEL AS FILES

5.1 Command Execution and Raw Output

Storage partitioning, mount tables, and virtual kernel interfaces were inspected:

```
vitalie@LinuxLabs:~/os-lab5$ lsblk
NAME   MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
sda      8:0    0    42G  0 disk
+-sda1   8:1    0     1M  0 part
+-sda2   8:2    0    42G  0 part /

vitalie@LinuxLabs:~/os-lab5$ mount | grep " / "
/dev/sda2 on / type ext4 (rw,relatime)

vitalie@LinuxLabs:~/os-lab5$ df -hm /
Filesystem     1M-blocks  Used Available Use% Mounted on
/dev/sda2          42027  9159     30702  23% /

vitalie@LinuxLabs:~/os-lab5$ grep -c processor /proc/cpuinfo
4

vitalie@LinuxLabs:~/os-lab5$ head -3 /proc/meminfo
MemTotal:        3479648 kB
MemFree:          404992 kB
MemAvailable:    2035864 kB

vitalie@LinuxLabs:~/os-lab5$ head -5 /proc/self/status
Name:   head
Umask:  0002
State:  R (running)
```

5.2 Analysis and Observations

a) **Storage architecture:** The virtual system presents drive `sda` (42 GB capacity). Partition `sda2` is mounted at the root directory (`/`) using the **ext4** filesystem.

b) **Filesystem capacity:** The root partition is currently **23% full**, utilizing 9,159 MB out of 42,027 MB available.

c) **Hardware metrics exposed in `/proc`:**

- Processor cores: **4 cores**.
- Total system RAM: **3,479,648 kB** ($\approx$ 3.4 GB).

d) **Behavior of `/proc/self/status`:** The output displays `Name: head`. The virtual link `/proc/self` resolves dynamically to the PID of the calling process accessing the filesystem entry. Because `head` opened the status file, the kernel reported the metadata belonging to `head`.

6 STRETCH GOALS

6.1 Stretch Goal A: Magic Numbers

The file signature header of a gzip-compressed archive was examined:

```
vitalie@LinuxLabs:~/os-lab5$ gzip -c /etc/passwd > notes.txt ; file notes.txt ; xxd notes.txt | head -1
notes.txt: gzip compressed data, was "passwd", last modified: Mon Oct  5 13:09:51 2026, from Unix, original size modulo 2^32 2820
00000000: 1f8b 0808 9fa1 c36a 0003 7061 7373 7764  .......j..passwd
```

Analysis: The initial two hexadecimal bytes `1f 8b` represent the magic number identifying gzip streams according to RFC 1952. The `file` utility inspects these leading bytes against its magic database, allowing accurate file classification regardless of the file extension.

6.2 Stretch Goal C: Deletion of an Active Open File

The filesystem reaction to unlinking a file with an active descriptor was tested:

```
vitalie@LinuxLabs:~/os-lab5$ truncate -s 10M bigfile ; exec 3< bigfile ; rm bigfile ; ls -l /proc/$$/fd ; exec 3<&-
lr-x------ 1 vitalie vitalie 64 Oct 12 12:49 3 -> '/home/vitalie/os-lab5/bigfile␣(deleted)'
```

Analysis: File descriptor 3 retains an open link to the deleted inode. Although `rm` removed the directory namespace entry, the inode and its allocated space remain reserved until descriptor 3 is closed with `exec 3<&-`.

CONCLUSION

This laboratory work provided a direct, practical inspection of the Linux filesystem architecture and its underlying POSIX mechanics. Performing these experiments on a live Linux system demonstrated how userland commands map directly to kernel filesystem abstractions.

The most important insights gained from this laboratory work are:

- **Separation of names and data:** Directory entries are merely names mapped to inode pointers; file metadata and data blocks exist independently of the directory entries that reference them.
- **Link mechanics:** Hard links share identical inode structures directly, while symbolic links are separate files storing destination string paths.
- **Deferred inode reclamation:** Unlinking a file only removes its directory entry; actual data blocks are only returned to the free list when the link count reaches zero and all processes close open file descriptors.
- **Permission semantics:** Directory permissions require both read (`r`) to list file names and execute (`x`) to traverse into subdirectories and resolve inodes.
- **Kernel exposure via pseudo-filesystems:** Virtual filesystems such as `/proc` demonstrate the UNIX concept of presenting live hardware configurations and process states through regular file read operations.

Overall, this practical work bridged theoretical knowledge of filesystem design with actual operating system behavior, reinforcing how file storage, security checks, and process resources are managed by the Linux kernel.

REFERENCES

[1] Robert Love, *Linux System Programming: Talking Directly to the Kernel and C Library*, O'Reilly Media.

[2] Michael Kerrisk, *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*, No Starch Press.

[3] Linux Kernel Documentation, *The /proc Filesystem*, https://docs.kernel.org/filesystems/proc.html

[4] POSIX IEEE Std 1003.1-2017, *Standard for Information Technology—Portable Operating System Interface (POSIX)*, File Systems and Permissions.